

Application Security Test Cases

Enterprise DevSecOps Security Lab

Author: Jagriti Banerjee

Document Classification: Portfolio / Private Lab Evidence

A detailed application security test-case catalog for the intentionally vulnerable Flask application used in the Enterprise DevSecOps project. The document defines test objectives, execution steps, expected results, evidence mapping, remediation validation, and CI/CD integration points.

33

Defined test cases

5

Core vulnerable routes

6

Primary risk areas

3

CI/CD AppSec checks

Scope: SQL Injection - Command Injection - Template Injection - Input Validation - Security Headers - SAST - Dependency Audit - CI/CD Evidence

1. Document Overview

This document defines a structured application security test plan and test-case catalog for the Enterprise DevSecOps Security Lab. The goal is to show how application-level weaknesses were identified, validated, documented, and mapped to remediation requirements inside a controlled private lab environment.

The application under test is a deliberately vulnerable Python Flask service. It contains intentionally unsafe code paths for SQL injection, command injection, and template rendering tests. These weaknesses are used only for private lab demonstration and professional reporting practice.

Project	Enterprise DevSecOps Security Lab
Document Type	Application Security Test Cases and Validation Plan
Author	Jagriti Banerjee
Application Under Test	Python Flask intentionally vulnerable demo application
Primary Endpoints	/health, /, /user, /ping, /hello
Testing Focus	Input validation, injection, dependency risks, static analysis, CI/CD AppSec evidence
Execution Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions evidence

Testing Principles

- Execute tests only against the local lab application and private VM environment.
- Document both vulnerable behavior and expected secure behavior for retesting.
- Treat proof-of-concept payloads as validation evidence, not exploitation against real systems.
- Map each finding to a remediation pattern and validation step.
- Integrate SAST and dependency checks into CI/CD so test evidence can be repeated.

2. Test Coverage Summary

The AppSec test suite is designed to cover both manual validation and automated pipeline evidence. Manual tests focus on endpoint behavior and proof-of-concept validation. Automated tests focus on repeatable detection through Bandit, pip-audit, and CI/CD workflow evidence.

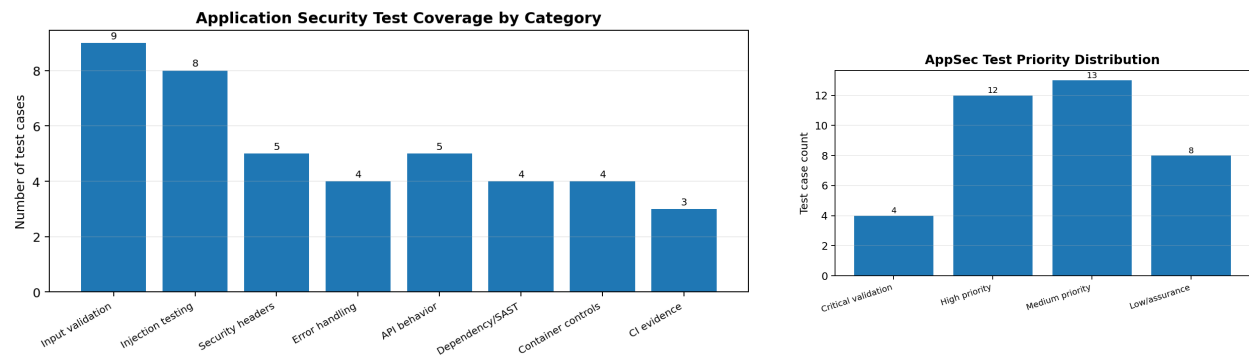


Figure 1. Application security coverage and priority distribution.

Endpoint Inventory

Endpoint	Method	Purpose	Security Testing Focus
/health	GET	Health status endpoint	Availability validation; baseline response integrity
/	GET	Landing page listing test routes	Information disclosure review and route discovery
/user?id=	GET	SQLite-backed user lookup	SQL injection and input validation testing
/ping?host=	GET	Ping utility using subprocess	Command injection and OS command execution testing
/hello?name=	GET	Template rendering route	Server-side template injection style testing

Risk Overview

Area	Primary Risk	Business/Security Impact	Priority
SQL Injection	Dynamic SQL query construction with user-controlled id	Unauthorized data access and query manipulation in the lab database	High
Command Injection	shell=True with untrusted host parameter	Arbitrary command execution inside application container/host context	Critical
Template Injection	User input inserted into template string	Server-side expression evaluation and potential information exposure	High
Dependency Exposure	Known vulnerable packages in requirements.txt	Exploitability through vulnerable transitive or direct packages	High
Security Headers	Missing browser defense headers	Increased client-side attack surface	Medium

3. Execution Methodology

The testing methodology follows a repeatable validate-document-remediate-retest cycle. Each test case contains a unique identifier, objective, target route, test data, expected result, and status. Where possible, the test is supported by screenshot evidence or tool output from the lab repository.

Step	Activity	Purpose
1	Baseline availability check	Confirm the Flask service is reachable before security testing.
2	Endpoint inventory	Identify application routes and expected behavior.
3	Input validation testing	Submit valid, invalid, malformed, encoded, and oversized inputs.
4	Injection validation	Test SQL, command, and template injection style payloads in the controlled lab.
5	Automated analysis	Run Bandit and pip-audit locally and in CI/CD.
6	Evidence capture	Save screenshots, JSON reports, and terminal outputs.
7	Remediation mapping	Define code-level fixes and secure configuration changes.
8	Retest criteria	Define how each weakness should be validated after remediation.

Local Test Harness

The following local commands represent the expected private-lab test harness. They are documented for repeatability and should only be executed inside the project VM or container environment.

```
python3 -m venv .envv
source .envv/bin/activate
pip install -r src/app/requirements.txt
python src/app/init_db.py
python src/app/app.py

curl http://localhost:5000/health
curl "http://localhost:5000/user?id=1"
curl "http://localhost:5000/ping?host=127.0.0.1"
curl "http://localhost:5000/hello?name=Jagriti"
```

4. Detailed Application Security Test Cases

The following table provides the formal test-case catalog. The test status reflects the intentionally vulnerable lab state where applicable. Items marked as planned represent validation checks that should be executed after remediation.

ID	Category	Target	Objective	Test Data / Command	Expected Result
APP-TC-001	Availability	/health	GET health endpoint with no parameters	curl http://localhost:5000/health	HTTP 200 JSON with service/status fields; no stack trace
APP-TC-002	Route inventory	/	GET landing page	curl http://localhost:5000/	Landing page visible; routes documented; no sensitive secrets disclosed
APP-TC-003	SQLi baseline	/user?id=1	Valid numeric lookup	curl "http://localhost:5000/user?id=1"	Single expected user returned; query_used field confirms test path
APP-TC-004	SQL Injection	/user?id=	Boolean-based payload	id=1' OR '1'='1	Multiple users returned, confirming SQL injection vulnerability in lab
APP-TC-005	SQL Injection	/user?id=	Quote handling / syntax pressure	id=1'	Application should reject invalid input gracefully; current behavior may expose query behavior
APP-TC-006	SQL Injection	/user?id=	Comment-style payload	id=1'--	Should not alter query logic; parameterized queries required
APP-TC-007	SQLi remediation check	/user?id=	Parameterized query retest	Same payloads after fix	Payloads treated as data; only intended row or validation error returned
APP-TC-008	Command baseline	/ping?host=127.0.0.1	Valid host value	curl "http://localhost:5000/ping?host=127.0.0.1"	Ping executes for allowed host; no extra command output
APP-TC-009	Command Injection	/ping?host=	Shell metacharacter injection	host=127.0.0.1;whoami	whoami output appears in response, confirming lab vulnerability
APP-TC-010	Command Injection	/ping?host=	Command separator variant	host=127.0.0.1%26%26id	Extra OS command must not execute; current shell=True risk requires fix
APP-TC-011	Command Injection	/ping?host=	Pipe/redirect metacharacters	host=127.0.0.1 id	Should reject; validate with allowlist and shell=False
APP-TC-012	Command remediation	/ping?host=	Retest after shell=False	127.0.0.1;whoami	Payload treated as invalid host; no OS command output
APP-TC-013	Template baseline	/hello?name=Jagriti	Normal name rendering	curl "http://localhost:5000/hello?name=Jagriti"	Expected greeting returned; no template evaluation
APP-TC-014	SSTI style	/hello?name=	Expression evaluation test	name={{7*7}} URL encoded	Application should render as text; if 49 appears, template injection exists
APP-TC-015	Input validation	/hello?name=	Long input string	5,000 character name	Application should cap length and return controlled response
APP-TC-016	Input validation	/user?id=	Non-numeric input	id=abc	Return 400 validation error; do not build SQL query with string
APP-TC-017	Input validation	/ping?host=	Invalid host format	host=not a real host	Return controlled error; avoid shell invocation
APP-TC-018	Security headers	All responses	Header review	curl -I http://localhost:5000/	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy recommended
APP-TC-019	Error handling	All routes	Malformed requests	Invalid payloads across endpoints	No debug traceback or internal file paths returned
APP-TC-020	Debug mode	Runtime config	Verify Flask debug behavior	Check FLASK_DEBUG and app startup	Debug mode must remain disabled outside controlled local testing
APP-TC-021	SAST	src/app	Bandit recursive scan	bandit -r src/app	Bandit reports subprocess/shell and template/string issues

ID	Category	Target	Objective	Test Data / Command	Expected Result
APP-TC-022	Dependency audit	requirements.txt	pip-audit scan	pip-audit -r src/app/requirements.txt	Known vulnerable dependencies identified and prioritized
APP-TC-023	Container runtime	Docker image	Non-root execution check	docker run --entrypoint id image	Container should run as UID 10001, not root
APP-TC-024	Kubernetes probes	Deployment	Health/readiness check	kubectl describe deployment demo-app	Readiness and liveness probes use /health endpoint
APP-TC-025	CI/CD AppSec	GitHub Actions	Pipeline evidence review	Bandit + pip-audit jobs	AppSec checks run automatically and publish reports

5. Negative and Edge Test Cases

Negative testing verifies that the application fails safely. These tests are especially important for AppSec because real attackers often use malformed, encoded, incomplete, or oversized inputs rather than normal functional inputs.

ID	Type	Target	Input Condition	Expected Secure Behavior
APP-NEG-001	Missing parameter	/user	No id supplied	Default handling should be explicit; prefer 400 for missing required parameter
APP-NEG-002	Empty parameter	/user?id=	Empty value	Return validation error instead of SQL query execution
APP-NEG-003	URL encoding	/ping?host=	Encoded semicolon	Encoded metacharacters should still be rejected
APP-NEG-004	Whitespace injection	/ping?host=	Host with spaces	Reject unexpected whitespace and shell tokens
APP-NEG-005	HTML injection	/hello?name=	<script>alert(1)</script>	HTML must be escaped; no script execution in browser
APP-NEG-006	Large response	/ping?host=	Long or slow command input	Server should not hang or consume excessive resources
APP-NEG-007	Unsupported method	/user	POST instead of GET	Return controlled 405; no stack trace
APP-NEG-008	Content type	All routes	Unexpected Accept header	Response should remain consistent and safe

Pass/Fail Interpretation

- Pass means the behavior matched the expected secure or baseline behavior in the lab.
- Fail - Vulnerable means the vulnerable condition was reproduced intentionally for evidence.
- Needs improvement means the control should be strengthened even if the app did not crash.
- Planned means the test should be rerun after code remediation is implemented.

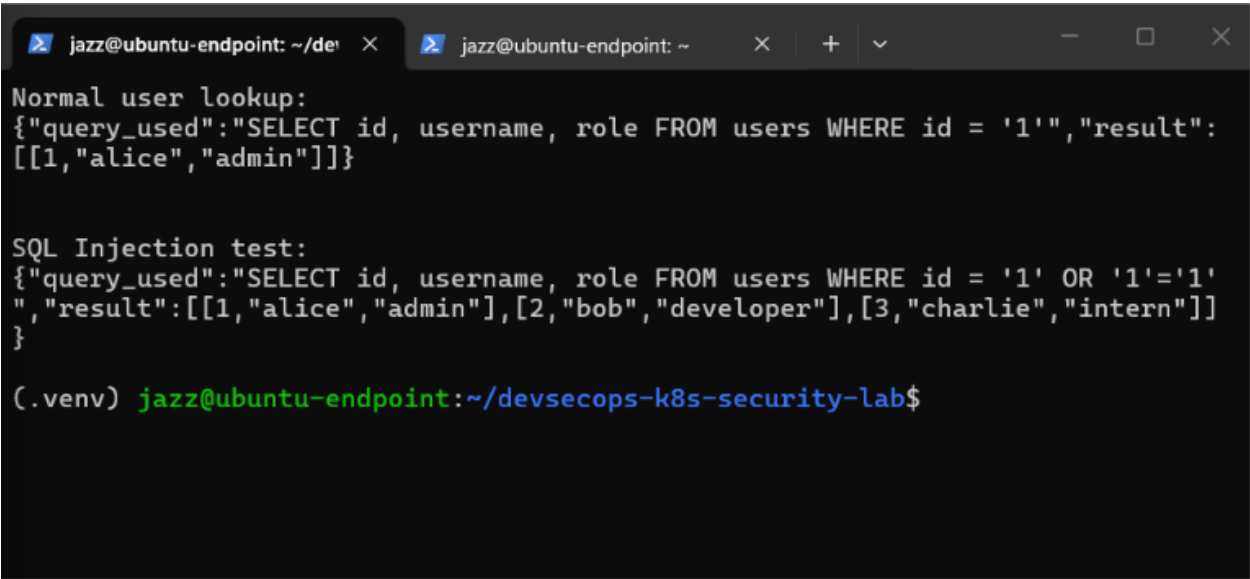
6. Injection Test Details and Evidence

6.1 SQL Injection Test Case Detail

The /user endpoint receives an id parameter and uses it to build a SQL query. In the lab implementation, the parameter is inserted into the query string directly. This allows boolean-based SQL injection payloads to change query logic.

```
curl "http://localhost:5000/user?id=1%27%20OR%20%271%27%3D%271"
```

Validation Point	Expected Secure Result	Observed Lab Result
Payload Handling	Payload should be treated as data or rejected with 400.	Multiple users are returned, confirming SQL injection behavior.
Root Cause	Parameterized queries should be used.	Dynamic f-string SQL query construction.
Remediation Retest	Same payload should not change query logic.	Retest after parameterized query implementation.



```
jazz@ubuntu-endpoint: ~/de  x  jazz@ubuntu-endpoint: ~  x  +  v  -  □  x

Normal user lookup:
{"query_used":"SELECT id, username, role FROM users WHERE id = '1',"result":
[[1,"alice","admin"]]}

SQL Injection test:
{"query_used":"SELECT id, username, role FROM users WHERE id = '1' OR '1'='1'
","result":[[1,"alice","admin"],[2,"bob","developer"],[3,"charlie","intern"]]}

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 2. SQL injection proof captured from private lab testing.

6.2 Command Injection Test Case Detail

The /ping endpoint uses the host parameter inside a shell command. Because subprocess.run is called with shell=True, shell metacharacters can be used to append additional commands in the lab environment.

```
curl "http://localhost:5000/ping?host=127.0.0.1%3Bwhoami"
```

Validation Point	Expected Secure Result	Observed Lab Result
Command Separator	Semicolon must be rejected or treated as invalid input.	Additional command output appears, confirming command injection.
Root Cause	Use shell=False and argument list.	shell=True with untrusted user input.
Remediation Retest	Injected commands should not execute.	Retest with ;whoami, &id, id, and encoded variants.

```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ curl "http://localhost:5000/ping?host=127.0.0.1%3Bwhoami"
Normal ping test:
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.154 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 1ms
rtt min/avg/max/mdev = 0.154/0.154/0.154/0.000 ms
</pre>

Command injection test with id:
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.063 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.063/0.063/0.063/0.000 ms
uid=1000(jazz) gid=1000(jazz) groups=1000(jazz),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lxd),999(docker)
</pre>

Command injection test with whoami:
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.060 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.060/0.060/0.060/0.000 ms
jazz
</pre>

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$ curl "http://localhost:5000/ping?host=127.0.0.1%3Becho%2BCOMMAND_INJECTION_WORKED"
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.062 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.062/0.062/0.062/0.000 ms
COMMAND_INJECTION_WORKED
</pre>

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

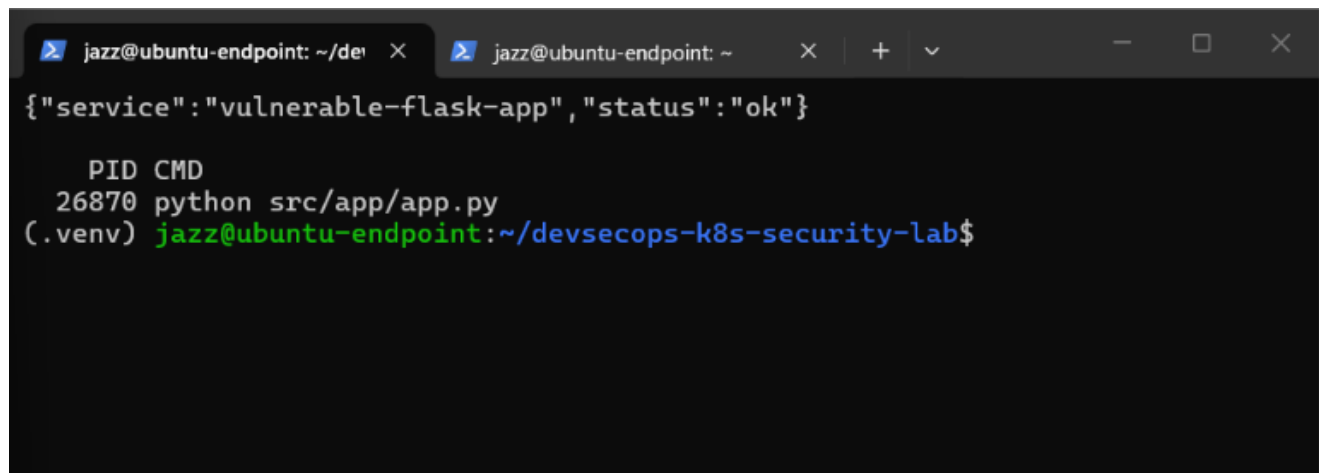
Figure 3. Command injection proof captured from private lab testing.

6.3 Template Injection Style Test Detail

The /hello endpoint builds a template string using user-controlled input and passes it to `render_template_string`. This is unsafe because user input can be interpreted as template syntax when it is not handled as plain text.

```
curl "http://localhost:5000/hello?name=%7B%7B7*7%7D%7D"
```

Validation Point	Expected Secure Result	Risk if Vulnerable
Expression Handling	The string <code>{{7*7}}</code> should be displayed literally.	If 49 appears, user input is being evaluated as template syntax.
Root Cause	Avoid dynamic template construction using user input.	<code>render_template_string</code> receives an f-string constructed from request input.
Remediation Retest	Use static template with escaped variable.	Payload should render as harmless text.



```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ curl "http://localhost:5000/hello?name=%7B%7B7*7%7D%7D"
{"service": "vulnerable-flask-app", "status": "ok"}

PID CMD
26870 python src/app/app.py
(.venv) jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Figure 4. Vulnerable Flask application running locally with test routes available.

7. Automated AppSec Validation

7.1 Bandit SAST Test Cases

Bandit was used to scan the Python source code for risky patterns. In this project, Bandit supports the manual findings by identifying unsafe subprocess usage and related security concerns in the application code.

```
bandit -r src/app -f json -o security/reports/bandit-report.json || true
bandit -r src/app || true
```

Test ID	Tool	Target	Expected Evidence
APP-SAST-001	Bandit	src/app/app.py	Subprocess and shell-related risk indicators are reported.
APP-SAST-002	Bandit JSON	security/reports/bandit-report.json	Machine-readable report is generated for CI/CD evidence.
APP-SAST-003	CI job	GitHub Actions	Bandit job runs automatically and uploads report artifact.

```

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b682_subprocess_
popen_with_shell_equals_true.html
Location: src/app/app.py:55:13
54         f"ping -c 1 {host}",
55         shell=True,
56         capture_output=True,
57         text=True
58     )
59     return f"<pre>{result.stdout}\n{result.stderr}</pre>"
60
61
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interf
aces.
Severity: Medium Confidence: Medium
CWE: CWE-695 (https://cwe.mitre.org/data/definitions/695.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104_hardcoded_b
ind_all_interfaces.html
Location: src/app/app.py:73:17
72     debug_mode = os.getenv("FLASK_DEBUG", "false").lower() == "true"
73     app.run(host="0.0.0.0", port=5000, debug=debug_mode)

Code scanned:
Total lines of code: 79
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
  Undefined: 0
  Low: 1
  Medium: 2
  High: 1
Total issues (by confidence):
  Undefined: 0
  Low: 1
  Medium: 1
  High: 2
Files skipped (0):
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
  
```

Figure 5. Local Bandit SAST findings from the private lab.

7.2 Dependency Audit Test Cases

pip-audit was used to identify known vulnerable dependencies from requirements.txt. This validates the software composition analysis portion of the AppSec test plan and ensures vulnerable packages are identified before deployment.

```
pip-audit -r src/app/requirements.txt -f json -o security/reports/pip-audit-report.json || true
pip-audit -r src/app/requirements.txt || true
```

Test ID	Tool	Target	Expected Evidence
APP-SCA-001	pip-audit	src/app/requirements.txt	Known vulnerable packages are detected and listed.
APP-SCA-002	pip-audit JSON	security/reports/pip-audit-report.json	Machine-readable dependency report is generated.
APP-SCA-003	CI job	GitHub Actions	pip-audit runs automatically and uploads report artifact.

```

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ pip-audit -r src/app/requirements.txt -f json -o security/reports/pip-audit-report.json || true
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ pip-audit -r src/app/requirements.txt || true

Found 25 known vulnerabilities in 5 packages
-----
Name      Version ID      Fix Versions
-----
flask     2.3.3          CVE-2026-27285  3.1.3
requests  2.19.1         PYSEC-2019-28   2.20.0
requests  2.19.1         PYSEC-2023-74   2.31.0
requests  2.19.1         PYSEC-2023-74   2.31.0
requests  2.19.1         CVE-2024-35195  2.32.0
requests  2.19.1         CVE-2024-87881  2.32.4
requests  2.19.1         CVE-2024-25685  2.33.0
gunicorn  21.2.0         CVE-2024-1135   22.0.0
gunicorn  21.2.0         CVE-2024-6827   22.0.0
idna      2.7            PYSEC-2024-60   3.7
idna      2.7            PYSEC-2024-60   3.7
idna      2.7            CVE-2026-45409  3.15
urllib3   1.23           PYSEC-2019-133  1.24.2
urllib3   1.23           PYSEC-2019-132  1.24.3
urllib3   1.23           PYSEC-2020-140  1.25.9
urllib3   1.23           PYSEC-2023-192  1.26.17, 2.0.6
urllib3   1.23           PYSEC-2023-192  1.26.17, 2.0.6
urllib3   1.23           PYSEC-2023-212  1.26.18, 2.0.7
urllib3   1.23           PYSEC-2023-207  1.24.2
urllib3   1.23           PYSEC-2023-212  1.26.18, 2.0.7
urllib3   1.23           PYSEC-2026-141  2.7.0
urllib3   1.23           CVE-2024-37891  1.26.19, 2.2.2
urllib3   1.23           CVE-2025-58181  2.8.0
urllib3   1.23           CVE-2025-66471  2.6.0
urllib3   1.23           CVE-2026-21441  2.6.3
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Figure 6. pip-audit dependency findings from the private lab.

8. CI/CD AppSec Evidence Requirements

Application security testing should not remain a one-time manual activity. The same checks should run inside CI/CD so that risky code changes and vulnerable dependency updates are detected before deployment.

Control	Pipeline Evidence	Acceptance Criteria
SAST	Bandit job output and JSON artifact	Job executes on push or pull request; report is retained.
SCA	pip-audit job output and JSON artifact	Dependency vulnerabilities are visible in pipeline logs/artifacts.
Container/Image Scan	Trivy scan artifact	Image findings are collected before release.
Manual AppSec Evidence	Screenshots and findings report	Proof-of-concept validation is documented without exposing real systems.
Retest Evidence	Before/after screenshots and updated reports	Remediation can be verified with repeatable commands.

```

jazz@ubuntu-endpoint: ~/jen
=====
33 - GitHub Actions / CI-CD Evidence
=====
File: .github/workflows/devsecops-pipeline.yml
Purpose: Local CI/CD workflow evidence for API tests, SAST, secrets, Trivy,
Docker, and SBOM
Note: Local workflow file only; no git push or GitHub repo actions performed
=====

[1] Workflow File Created:
-rw-rw-r-- 1 jazz jazz 2.2K Jun 12 22:28 .github/workflows/devsecops-pipeline.yml

[2] Pipeline Jobs:
push:
pull_request:
workflow_dispatch:
api-tests:
sast:
secrets:
container-scan:
sbom:

[3] Security Tools Included:
  pip install fastapi "uvicorn[standard]" pytest httpx
  PYTHONPATH=$PWD pytest -v tests/test_api.py
  uses: sengrep/sengrep-action@v1
  config: security-scans/rules/sengrep-api-security.yml
  uses: gitLeaks/gitLeaks-action@v2
  docker build -t enterprise-devsecops-banking-api:ci .
  uses: aquasecurity/trivy-action@master
  sbom:
    docker build -f Dockerfile.secure.docs -t enterprise-devsecops-banking-api-secure:ci
    uses: anchore/sbom-action@v0

=====
Evidence: CI/CD pipeline workflow documented locally successfully
=====
jazz@ubuntu-endpoint: ~/enterprise-devsecops-redteam-lab$

```

Figure 7. CI/CD workflow evidence demonstrating repeatable AppSec checks.

9. Remediation and Retest Matrix

Each AppSec issue should be remediated with a code or configuration change and then retested using the same test case that originally reproduced the issue. This creates clear proof that the defect moved from vulnerable to fixed.

Finding Area	Recommended Remediation	Retest Criteria
SQL Injection	Use parameterized SQLite queries; reject non-numeric id values; return 400 for invalid input.	Retest APP-TC-004 to APP-TC-007 and confirm payloads are treated as data.
Command Injection	Remove shell=True; call subprocess with argument list; validate host with ipaddress/allowlist.	Retest APP-TC-009 to APP-TC-012 and confirm no additional command output.
Template Injection	Avoid f-string construction inside render_template_string; use static templates and escaped variables.	Retest APP-TC-014 and confirm {{7*7}} renders as literal text, not 49.
Security Headers	Add Flask-Talisman or reverse-proxy headers: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy.	Retest APP-TC-018 with curl -I and verify headers are present.
Dependency Risk	Upgrade vulnerable packages, pin versions, and keep pip-audit in CI/CD.	Re-run pip-audit and verify advisory count decreases or is accepted with justification.
SAST Findings	Refactor unsafe subprocess/template patterns and add secure coding review gates.	Re-run Bandit and confirm high-confidence issues are closed or documented.

Secure Coding Patterns

Unsafe Pattern	Secure Replacement
String-built SQL query	Parameterized query using placeholders.
subprocess with shell=True	subprocess argument list with shell=False and strict allowlist.
render_template_string with user input	Static template file with escaped variable interpolation.
Unbounded parameter values	Length, type, and format validation before processing.
No security headers	Add response security headers through Flask middleware or reverse proxy.

10. OWASP and Portfolio Mapping

The test cases are mapped to widely recognized application security themes so the project can be reviewed by recruiters, security engineers, and interviewers as a professional AppSec portfolio artifact.

OWASP Theme	Project Coverage	Related Test IDs
A01 Broken Access Control	No direct auth model in this lab; validate unauthenticated exposure and endpoint minimization.	APP-TC-002, APP-TC-019
A03 Injection	SQL injection, command injection, and template injection style tests.	APP-TC-004 to APP-TC-014
A05 Security Misconfiguration	Headers, debug mode, error handling, container runtime hardening.	APP-TC-018 to APP-TC-024
A06 Vulnerable and Outdated Components	pip-audit against requirements.txt; CI dependency scan evidence.	APP-TC-022, APP-TC-025
A09 Security Logging and Monitoring Failures	CI artifacts, scanner outputs, and runtime evidence integration.	APP-TC-021, APP-TC-025

Professional Review Notes

- The test cases demonstrate both vulnerability reproduction and secure retest planning.
- The document separates manual proof-of-concept testing from automated CI/CD evidence.
- Evidence screenshots support the validity of the AppSec test execution.
- The remediation matrix shows practical secure coding understanding, not only tool usage.

```

jazz@ubuntu-endpoint: ~/jen
[1] API Security Findings:
## API Security Findings

| Finding ID | Severity | Finding | Evidence |
|---|---|---|---|
| API-001 | HIGH | Exposed admin debug endpoint | '/api/v1/admin/debug' returned environment and database details |
| API-002 | HIGH | Hardcoded JWT secret | Semgrep detected 'JWT_SECRET' in 'app/main.py' |
| API-003 | MEDIUM | Hardcoded database URL | Semgrep detected PostgreSQL connection string in source code |
| API-004 | MEDIUM | Missing production-grade authentication | Demo bearer token used for protected user endpoint |

[2] Kubernetes Security Findings:
## Kubernetes Security Findings

| Finding ID | Severity | Finding | Evidence |
|---|---|---|---|
| K8S-001 | HIGH | Privileged container allowed before policy enforcement | Insecure deployment used 'privileged: true' |
| K8S-002 | HIGH | Privilege escalation allowed before policy enforcement | Insecure deployment used 'allowPrivilegeEscalation: true' |
| K8S-003 | HIGH | Root user execution allowed before policy enforcement | Insecure deployment used 'runAsUser: 0' |
| K8S-004 | PASS | Kyverno blocked insecure deployment | Admission webhook denied insecure workload |

[3] Final Status:
## Final Status

Docker images were built successfully.
API tests passed successfully.
SAST and secret scanning evidence was generated.
Trivy filesystem and image scans were completed.
SBOM was generated successfully.
Kubernetes secure deployment passed Kyverno policy enforcement.
Final validation completed successfully.

Evidence: API security findings report generated successfully
=====
jazz@ubuntu-endpoint: ~/enterprise-devsecops-redteam-lab$

```

Figure 8. API security findings evidence from the Enterprise DevSecOps lab.

11. Final AppSec Test Execution Checklist

Item	Completion Evidence
Application starts locally	Screenshot of Flask app running and health endpoint response.
Baseline endpoints tested	/health, /, /user, /ping, and /hello validated.
SQL injection reproduced	Boolean-based payload returns unintended users in lab.
Command injection reproduced	Injected command output appears in lab response.
Template injection style condition tested	Expression payload used against /hello route.
Bandit executed locally	Terminal output and JSON report captured.
pip-audit executed locally	Terminal output and JSON report captured.
CI/CD AppSec checks reviewed	GitHub Actions evidence captured.
Remediation plan documented	Fix patterns and retest criteria defined.
Evidence mapped	Screenshots referenced in the report and evidence index.

Conclusion

This AppSec test-case document provides a detailed and repeatable testing blueprint for the Enterprise DevSecOps Security Lab. It demonstrates how vulnerable behavior was safely validated in a private environment, how automated tools support manual findings, and how remediation should be verified through structured retesting.

Deliverable	Status
Application security test cases defined	Complete
Injection and input validation testing documented	Complete
SAST and dependency audit test coverage included	Complete
Remediation and retest criteria defined	Complete
Evidence screenshots referenced	Complete